INDIVIDUAL PROJECT REPORT

ON

"CampusConnect: Student Academic & Communication Portal"

INDIVIDUAL CONTRIBUTION REPORT — ACADEMICS & ASSIGNMENTS MODULE, DASHBOARD FRONTEND AND REPOSITORY INTEGRATION

Submitted in partial fulfillment of the requirement for the award of Degree of

Bachelor of Technology in Computer Science & Engineering

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted by

ADITYA GOUR

Enrollment No.: 24EARCS008

Individual responsibility: Academics & Assignments Module and Dashboard Frontend

Under the guidance of

Er. Ram Babu Buri

Department of Computer Science & Engineering

Session: 2026–27

ARYA COLLEGE OF ENGINEERING & I.T.

SP-42, RIICO Industrial Area, Kukas, Jaipur

CERTIFICATE

This is to certify that the individual project report entitled **"CampusConnect: Student Academic & Communication Portal — Academics & Assignments Module and Dashboard Frontend"** has been prepared by **Aditya Gour (24EARCS008)**, a student of Bachelor of Technology in Computer Science & Engineering at Arya College of Engineering & I.T., Jaipur, affiliated with Rajasthan Technical University, Kota, during the academic session 2026–27.

The report records the student's individual contribution to the group project CampusConnect, carried out under the guidance of Er. Ram Babu Buri. The contribution described here forms part of a five-member team project; the work of the other team members is acknowledged and is not claimed in this report.

The certificate wording, the implementation evidence and the final status of the work are subject to review by the department before the report is accepted.

Role	Name	Signature
Project Guide	Er. Ram Babu Buri Dept. of CSE, ACEIT, Jaipur	
Head of Department	Dr. Akhil Pandey Dept. of CSE, ACEIT, Jaipur	
Student	Aditya Gour (24EARCS008)	

Date: _____ Place: Jaipur

DECLARATION

I, **Aditya Gour (24EARCS008)**, a student of B.Tech. Computer Science & Engineering at Arya College of Engineering & I.T., Jaipur, hereby declare that this individual report describes my own contribution to the group project **CampusConnect: Student Academic & Communication Portal**, carried out during the session 2026–27 under the guidance of Er. Ram Babu Buri.

The technical content of this report was prepared from the project's source code and Git history as they existed on the `main` branch at commit `2572458` (11 October 2026), together with the official team project report. Every feature described as my work is traceable to commits recorded under my Git identity. Where a feature was built by a teammate, or where shared files were edited by more than one person, this is stated explicitly.

Test and build results quoted in this report were obtained by running the project's existing test suite and production build on 11 October 2026. Behaviour that has not been verified is labelled as pending. The report does not claim sole authorship of CampusConnect, measured performance results, or a production deployment.

To the best of my knowledge, this report has not been submitted to any other university or institution for the award of any degree or diploma.

Aditya Gour
Enrollment No.: 24EARCS008
B.Tech. (CSE), Session 2026–27

Signature: _____ Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **Er. Ram Babu Buri**, for his guidance throughout the planning and development of CampusConnect. His emphasis on clear requirements, honest reporting of project status and systematic verification shaped the way I designed and documented the Academics & Assignments module.

I am thankful to **Dr. Akhil Pandey**, Head of the Department of Computer Science & Engineering, Arya College of Engineering & I.T., for providing the academic environment and resources that made this project possible, and to the faculty members of the department for their support and feedback.

I am grateful to my teammates — **Anjali Kumari**, **Aayushman Pathak**, **Akshat Varma** and **Aman Kumar Chauhan**. My module depends directly on their work: the authentication and profile services provide the identity and student details on which academic access rules are based, while the chat, community, announcements and clubs modules share the dashboard shell that I built. Reviewing and merging their pull requests taught me a great deal about integration, version control and communication within a team.

Finally, I thank my family and friends for their constant encouragement and patience during the project.

Aditya Gour
Enrollment No.: 24EARCS008
Department of Computer Science & Engineering

ABSTRACT

CampusConnect is a web-based Student Academic and Communication Portal developed by a team of five students at Arya College of Engineering & I.T., Jaipur. It combines a React single-page application with a Java 21 / Spring Boot REST backend and a MySQL database, and is organised into modules for authentication and profiles, academics, announcements, clubs and events, and community chat.

This individual report documents my contribution to the project. My principal responsibility was the **Academics & Assignments module**. I first delivered a role-aware academics user interface backed by clearly labelled demonstration data, and then implemented the full stack: three new MySQL tables (subjects, assignments, submissions), JPA entities, repositories, a central access-control service, a file-storage service, three REST controllers exposing 22 endpoints, and an Axios-based client service. The module lets teachers create subjects and draft, publish, grade and return assignments; lets students see only the subjects that match their department, course, year and section and submit or revise work before the deadline; and gives administrators read-only oversight together with subject management.

I also built the shared **dashboard shell** used by every signed-in page — sidebar, header, mobile navigation, light/dark theme, route guards and role-specific Student, Teacher and Admin dashboards — as well as the public landing page. As the owner of the GitHub repository, I created the initial project scaffold, merged eleven pull requests, resolved merge conflicts between modules, removed hard-coded default secrets, and prepared a multi-stage Docker build that serves the frontend and API from a single Spring Boot application.

Verification was carried out on 11 October 2026: the backend test suite ran 227 tests with no failures (104 of them in the academics test classes I wrote), and the production frontend build completed successfully. End-to-end testing against a live MySQL database, automated frontend tests and the live data source for dashboard assignment widgets remain pending and are reported as limitations.

Keywords: React, Spring Boot, REST API, role-based access control, assignment management, file upload security, Git collaboration.

TABLE OF CONTENTS

Certificate 2
Declaration 3
Acknowledgement 4
Abstract 5
List of Abbreviations 7
CHAPTER 1 — INTRODUCTION 8
CHAPTER 2 — PROJECT OVERVIEW, SCOPE AND INDIVIDUAL RESPONSIBILITIES 9
CHAPTER 3 — TECHNOLOGY STACK AND DEVELOPMENT ENVIRONMENT 10
CHAPTER 4 — REQUIREMENTS ANALYSIS 11
CHAPTER 5 — SYSTEM ARCHITECTURE AND DESIGN 12
CHAPTER 6 — ACADEMICS & ASSIGNMENTS MODULE: DESIGN AND IMPLEMENTATION 14
CHAPTER 7 — FRONTEND, DASHBOARD AND USER INTERFACE CONTRIBUTIONS 16
CHAPTER 8 — API INTEGRATION AND BACKEND COORDINATION 18
CHAPTER 9 — GIT WORKFLOW, BRANCHING, MERGING AND TEAM COLLABORATION 20
CHAPTER 10 — TESTING, DEBUGGING AND VERIFICATION 21
CHAPTER 11 — IMPLEMENTATION RESULTS, CURRENT STATUS AND LIMITATIONS 22
CHAPTER 12 — CHALLENGES AND LEARNING OUTCOMES 23
CHAPTER 13 — FUTURE SCOPE AND CONCLUSION 24
REFERENCES 25
APPENDIX A — COMMIT EVIDENCE AND VERIFICATION LOG 26

List of Figures

Figure 5.1 — Layered architecture of CampusConnect with my contribution highlighted 12
Figure 5.2 — Entity–relationship diagram of the academics data model 13
Figure 6.1 — Use-case diagram of the Academics & Assignments module 14
Figure 6.2 — State diagrams for assignments and submissions 15
Figure 6.3 — Sequence diagram: a student submits an assignment 16
Figure 7.1 — Route, layout and component structure of the signed-in frontend 17
Figure 7.2 — Student dashboard (development demo mode) 18
Figure 9.1 — Branch and pull-request timeline, 8–11 October 2026 20
Figure 11.1 — Academics page, student view (development demo mode) 22
Figure 11.2 — Academics page, teacher assignments tab (development demo mode) 22
Figure A.1 — Academics page at a 390 px mobile viewport (development demo mode) 26

LIST OF ABBREVIATIONS

Term	Meaning
API	Application Programming Interface
ARIA	Accessible Rich Internet Applications (W3C accessibility attributes)
BCrypt	Adaptive password-hashing function used by the authentication module
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object (request/response type at the API boundary)
ER	Entity–Relationship (data model diagram)
FK / PK / UK	Foreign Key / Primary Key / Unique Key
FR / NFR	Functional Requirement / Non-Functional Requirement
HTTP(S)	Hypertext Transfer Protocol (Secure)
JPA	Jakarta Persistence API (object–relational mapping standard)
JSON	JavaScript Object Notation
JWT	JSON Web Token (RFC 7519)
MVC	Model–View–Controller
PR	Pull Request (GitHub)
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single-Page Application
SQL	Structured Query Language
UI / UX	User Interface / User Experience
URL	Uniform Resource Locator
UUID	Universally Unique Identifier (used for stored file names)

CHAPTER 1 – INTRODUCTION

1.1 Background

In many colleges, coursework is still distributed through a mixture of messaging groups, e-mail, printed notices and verbal reminders. Students lose track of deadlines, teachers collect submissions in different formats, and nobody has a reliable record of who has submitted what. CampusConnect, the group project described in the official team report, aims to bring authentication, academic work, announcements, clubs and community communication into a single portal for Arya College of Engineering & I.T., Jaipur.

The team report deliberately distinguishes between features that have been implemented and features that are planned. In particular, it lists assignments and the academic section as planned modules to be added after the authentication foundation is stable. My work picks up exactly at this point: I designed and built the Academics & Assignments module on top of the authentication and profile services developed by my teammate, and I built the dashboard shell into which every module is placed.

1.2 Purpose of this Report

This report documents my individual contribution in technical detail and, equally importantly, separates it from the contributions of my teammates. It explains what I designed, how the implementation works, how it fits into the wider system, how it was tested, and what is still incomplete. It is not a second copy of the team report; shared material is summarised only where it is needed to explain my module.

1.3 Problem Addressed by My Module

An assignment workflow is simple to describe but easy to get wrong. A student must only see the subjects and assignments intended for their class; a teacher must only manage the subjects they teach; an unpublished draft must stay invisible; a late or duplicate submission must be handled consistently; marks must stay hidden until the teacher releases them; and uploaded files must not become a security hole. My module therefore had to provide not just screens and tables, but a single, testable set of access and workflow rules enforced on the server.

1.4 Objectives of My Contribution

1. Provide a role-aware Academics & Assignments page for students, teachers and administrators.
2. Model subjects, assignments and submissions in MySQL with integrity constraints, and expose them through secured REST endpoints.
3. Centralise authorisation so that identity always comes from the authenticated user and never from the request body.
4. Support draft/publish, deadlines, optional late work, resubmission, grading, feedback and the controlled release of marks.
5. Accept file attachments and submissions safely, with type and size limits and clean-up on failure.
6. Provide a shared, responsive dashboard shell and role-specific dashboards that other modules can plug into.
7. Integrate teammates' work through the shared GitHub repository while keeping `main` buildable.

1.5 Method and Organisation

All technical statements were checked against the repository: the `main` branch, all local and remote branches, the full commit graph, the diffs of every commit recorded under my identity, and the 21 merged pull requests named in merge-commit messages. Tests and the production build were run on 11 October 2026 on isolated copies of the source. Chapters 2–5 cover context, technologies, requirements and architecture; Chapters 6–8 the academics module, user interface and API integration; Chapters 9–11 collaboration, testing and status; Chapters 12–13 learning outcomes, future scope and conclusion.

CHAPTER 2 – PROJECT OVERVIEW, SCOPE AND INDIVIDUAL RESPONSIBILITIES

2.1 Overall Project Context

CampusConnect is developed by five students — Aditya Gour, Anjali Kumari, Aayushman Pathak, Akshat Varma and Aman Kumar Chauhan — as an academic project. The repository is hosted on GitHub as `Adit841/CampusConnect` (my account), and work is integrated through feature branches and pull requests into `main`. The first release targets a single college; multi-college operation is a future direction in the team report.

2.2 Module Ownership as Recorded in Git

Module	Branch(es)	Authorship evidence in Git history
Authentication & profiles	`aman-auth-profile`	Commits by Aman Kumar Chauhan (e.g. `bb0ed69`, `22faf24`): JWT, BCrypt, login/register pages, token persistence.
Chat, community, clubs & events	`ayushman-feature`	Commits by Aayushman Pathak (e.g. `8e74450`, `c08b2fc`, `9bc5f28`).
Announcements	`anjali-announcements`	Implementation commit `d04672a` recorded under Aman Kumar Chauhan's Git identity; merged as PR #14.
Academics & assignments	`aditya/academics-assignments`, `akshat/assignment-academics`	Commits `22a5cf4` and `a53614c`, both recorded under my identity (see note below).
Dashboard shell, landing page, scaffold, integration	`aditya/frontend`, `aditya/fix-security-perf`, `aditya/registration-college`, `main`	Commits under my identity (Appendix A).

> **Attribution note.** The full-stack academics commit `a53614c` (69 files) was authored under my Git identity but on the branch `akshat/assignment-academics`, which is named after teammate Akshat Varma's academics work-stream, and was merged as PR #19. Git records no commits under Akshat Varma's or Anjali Kumari's identities. Any planning, review or testing they contributed outside Git cannot be measured from the repository and is acknowledged here rather than quantified.

2.3 In-Scope and Out-of-Scope Work

In scope: the complete Academics & Assignments module (tables, entities, repositories, services, controllers, DTOs, file storage, tests and frontend page); the shared dashboard shell, role-based dashboards, navigation, route guards and theme; the landing page and repository scaffold; and integration work — merging pull requests, resolving conflicts, removing default secrets, college-restricted registration and Docker packaging.

Out of scope: I do not claim the JWT filter, security filter chain, BCrypt configuration, login and registration pages, profile services, token storage, chat/WebSocket messaging, community feed, announcements, or clubs and events. Where I edited a teammate's file — adding a college field to registration or Arya clubs to the clubs seeder — it is described as a small contribution to their module.

Table 2.1 — Contribution matrix

Contribution	Evidence	Size of change	Status
Repository scaffold & docs	`3512458`, `e451da8`	33 files	Implemented
Dashboard shell & role dashboards	`3dce324` (PR #2)	43 files, +2,150 lines	Partial (mock data)
Academics UI (phase 1)	`22a5cf4` (PR #7)	15 files, +1,252 lines	Superseded by phase 2
Landing page	`9646c18`	15 files, +1,599 lines	Implemented
Academics full stack (phase 2)	`a53614c` (PR #19)	69 files, +6,632 lines	Implemented, unit-tested

Security & performance fixes	`ec9b9e2` (PR #18)	20 files	Implemented
College registration, clubs seed data	`9814db8`, `725fb65`	9 files	Implemented
Docker / single-origin deployment	`14860e5`, `c5678e9`, `2572458`	8 files	Deployment unverified

CHAPTER 3 – TECHNOLOGY STACK AND DEVELOPMENT ENVIRONMENT

3.1 Verified Technology Stack

The versions below were read directly from `backend/pom.xml`, `frontend/package.json`, the `Dockerfile` and the installed toolchain. Caret (^) versions are minimum compatible versions declared in `package.json`.

Table 3.1 — Technologies used in my contribution

Layer	Technology and version	How I used it
Frontend	React ^19.3.0, React Router ^8.4.0, lucide-react ^1.53.0	Pages, layouts, hooks, nested and role-guarded routes, icons
Build & styling	Vite ^8.3.4, Tailwind CSS ^4.3.3 (`@tailwindcss/vite`)	Dev server, production build, `React.lazy` chunks, responsive and dark-mode styles
HTTP client	Axios ^1.20.0	All academics API calls, multipart uploads, blob downloads
Backend language	Java 21 (Temurin 21.0.12 used for testing)	Records for DTOs, switch expressions, `getFirst()`
Backend framework	Spring Boot 4.1.1 (Web MVC, Data JPA, Validation, Security)	Controllers, services, repositories, `@PreAuthorize`, transactions
Authentication	Spring Security + JJWT 0.12.6 + BCrypt (team)	Consumed: the authenticated principal's e-mail identifies the user
Database	MySQL 8 (README prerequisite), Connector/J	Three new InnoDB tables with FK, UNIQUE and CHECK constraints
Testing	JUnit 5, Mockito, Spring MockMvc (spring-boot-starter-test)	8 academics test classes, 104 tests
Build & packaging	Maven 3.9, Docker multi-stage (node:22-alpine, maven:3.9-eclipse-temurin-21, eclipse-temurin:21-jre)	Single deployable JAR containing the built SPA
Version control	Git and GitHub	Feature branches, pull requests, merges, stash

The chat module additionally uses STOMP and SockJS over WebSocket; these belong to Aayushman Pathak's module and are not used by my code.

3.2 Development Environment

Development used macOS with the Temurin 21 JDK, Maven, Node.js and npm. Configuration is supplied through environment variables documented in `backend/.env.example` — database URL and credentials, `JWT_SECRET`, `ACADEMICS_STORAGE_DIR`, `ACADEMICS_MAX_FILE_MB` and allowed extensions; no credentials are reproduced here. The Vite dev server proxies `/api` and `/ws` to `localhost:8080` (configured by a teammate in `b8249b8`). The editor was Cursor: three of my commits (`22a5cf4`, `ec9b9e2`, `725fb65`) carry a `Co-authored-by: Cursor` trailer, recording that an AI-assisted editor was used while preparing those changes.

3.3 Rationale for Key Choices in My Module

Method-level security (`@PreAuthorize`) was used for coarse role checks because it keeps the rule beside the endpoint, while the finer ownership and eligibility checks live in one service so that they can be unit-tested without a web server. **Java records** keep DTOs immutable and concise. An injected `java.time.Clock` bean

(`AcademicsConfig`) makes deadline logic deterministic in tests. On the frontend, a single **service module** (`academicsService.js`) holds both the API calls and the display rules, so that components stay presentational and the rules can be reused by the dashboard.

CHAPTER 4 – REQUIREMENTS ANALYSIS

4.1 Users of the Academics Module

The module serves three roles defined by the authentication module's `Role` enumeration. **Students** consume coursework, **teachers** author and assess it, and **administrators** oversee it. Role is always read from the authenticated account on the server; the frontend uses it only to decide what to display.

4.2 Functional Requirements

Table 4.1 — Functional requirements of the Academics & Assignments module

ID	Requirement	Status
FR-A1	Teachers create and edit subjects (code, name, credits, department and optional course/year/section); admins may also create subjects and assign a teacher.	Implemented
FR-A2	Students see only subjects whose audience matches their profile.	Implemented
FR-A3	Teachers create assignments as drafts with title, instructions, maximum marks (1–1000) and a future deadline.	Implemented
FR-A4	Publishing is explicit and one-way; drafts are invisible to students.	Implemented
FR-A5	Teachers attach, replace or remove one reference file per assignment.	Implemented
FR-A6	Students submit a written response and/or a file once per assignment; late work is accepted only if allowed and is flagged.	Implemented
FR-A7	Students may revise a submission before the deadline while it is ungraded.	Implemented
FR-A8	Teachers see a roster of all eligible students, including non-submitters, filterable by pending/submitted/graded/late.	Implemented
FR-A9	Teachers grade within 0…max marks, add feedback, and return work; marks are hidden from students until returned.	Implemented
FR-A10	Assignments with submissions cannot be deleted or moved to another subject.	Implemented
FR-A11	Search, status filter, subject filter and sort on the assignment list.	Implemented
FR-A12	Dashboard deadline and review widgets use live academics data.	Partial — mock data
FR-A13	Notifications when assignments are published or graded.	Planned

4.3 Non-Functional Requirements

Table 4.2 — Non-functional requirements and how they are addressed

Category	Approach in the implementation
Security	Server-side role and ownership checks; hidden resources return 404; generated file names; extension allow-list; size limits.
Integrity	FK, UNIQUE and CHECK constraints in `schema.sql`; one submission per student per assignment enforced by a unique key.
Usability	Role-specific actions, inline field validation, clear empty/error states with retry, deep-linkable tabs.
Accessibility	ARIA tab roles, labelled inputs with `aria-invalid` / `aria-describedby`, skip link, visible focus rings.
Responsiveness	Mobile drawer navigation, stacked cards and filters at narrow widths (Figure A.1).
Maintainability	Controller–service–repository layering; one access-control service; DTO mapping isolated in `AcademicMapper`.
Testability	Injected `Clock`; services tested with Mockito; controllers with standalone MockMvc.

4.4 Constraints and Assumptions

Students are linked to subjects by matching profile fields rather than an explicit enrolment table, so a student whose profile lacks a department sees no subjects. Files are stored on the local file system of the server. The release targets one college, so subject codes are unique globally rather than per institution.

CHAPTER 5 – SYSTEM ARCHITECTURE AND DESIGN

5.1 Layered Architecture

CampusConnect follows the layered client–server architecture described in the team report and in `docs/architecture.md`, which I wrote as part of the initial scaffold. Figure 5.1 shows where my work sits in each layer. Requests from the React application pass through the shared Axios instance, the team's JWT filter and Spring Security, and reach my controllers only when the caller is authenticated; method-level rules then restrict each operation by role.

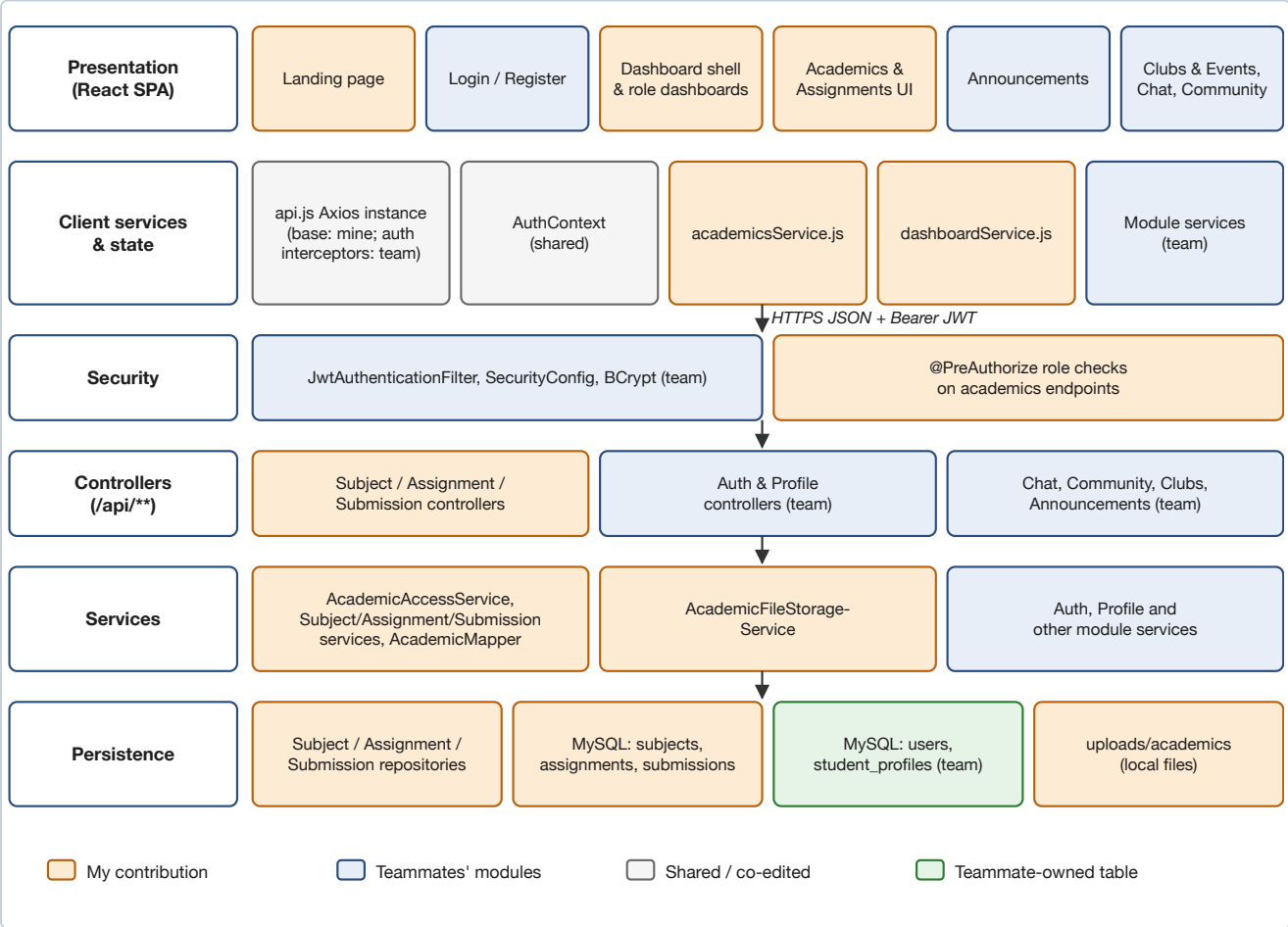


Figure 5.1 — Layered architecture of CampusConnect. Orange boxes are my contribution; blue boxes belong to teammates; grey boxes are shared files edited by several members.

5.2 Backend Structure of the Academics Module

Table 5.1 — Backend classes added in commit `a53614c`

Layer	Classes
Controllers	`SubjectController`, `AssignmentController`, `SubmissionController`
Services	`AcademicAccessService`, `SubjectService`, `AssignmentService`, `SubmissionService`, `AcademicFileStorageService`, `AcademicMapper`
Entities	`Subject`, `Assignment`, `Submission`, `StoredFile` (embeddable), `AssignmentStatus`, `SubmissionStatus`
Repositories	`SubjectRepository`, `AssignmentRepository`, `SubmissionRepository`; audience queries added to `StudentProfileRepository`

DTOs (records)	11 types, e.g. `AssignmentRequest`, `AssignmentResponse`, `GradeSubmissionRequest`, `SubmissionRosterEntry`
Support	`AcademicsConfig` (Clock), `ConflictException`, `FileDownloadResponses`, 409/413 handlers in `GlobalExceptionHandler`

5.3 Data Model

I added three tables to `database/schema.sql` (sections 8–10). They reference the `users` and `student_profiles` tables owned by the authentication module. Figure 5.2 shows the relationships and the most important columns.

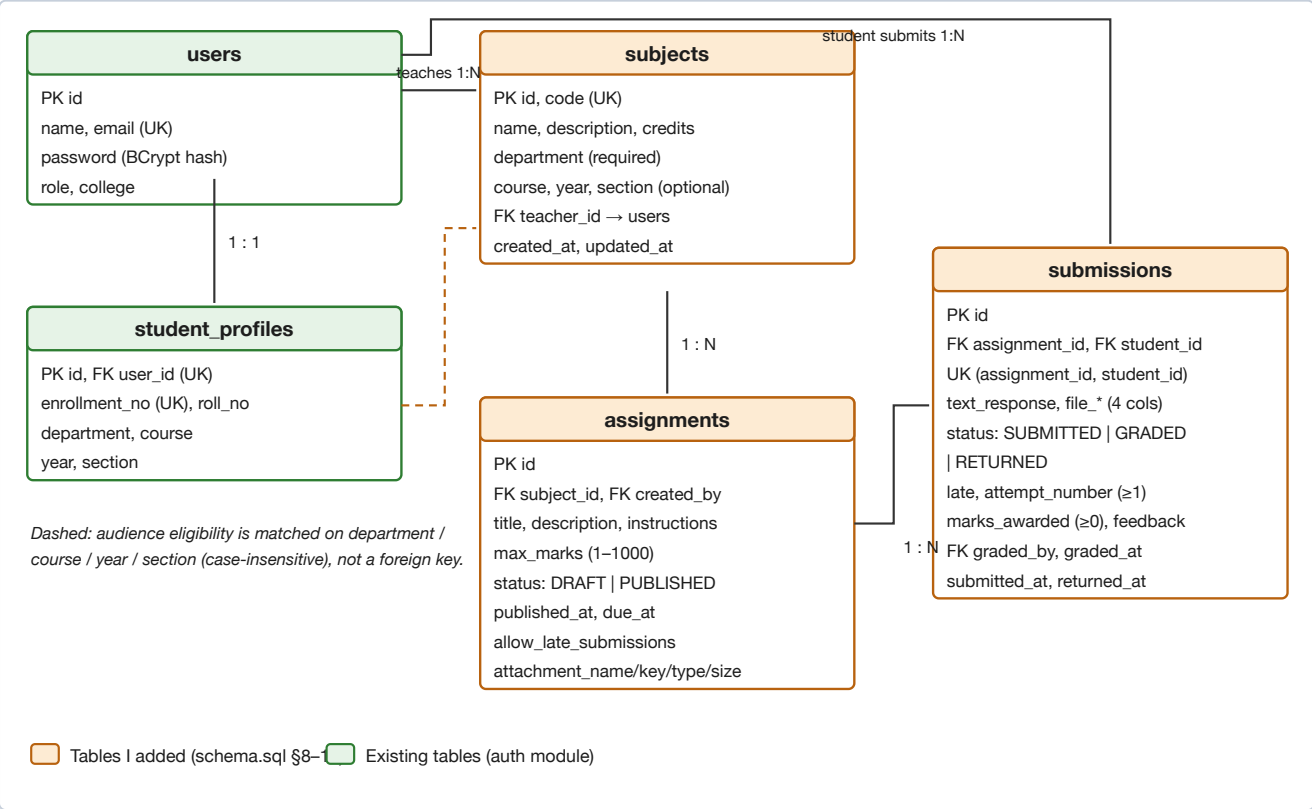


Figure 5.2 — Entity–relationship diagram of the academics data model, derived from `database/schema.sql`.

Integrity is enforced in the database as well as in code: subject codes are unique; credits must be 0–20 and year 1–10; `max_marks` must be 1–1000; status columns are restricted by `CHECK` constraints; `marks_awarded` cannot be negative; and `uk_submissions_assignment_student` guarantees one submission row per student per assignment. Composite indexes such as `(subject_id, status)` and `(assignment_id, status)` support the most frequent list queries.

5.4 Key Design Decisions

- **Identity from the principal.** Every service method receives the authenticated e-mail and loads the user itself; teacher or student identifiers in request bodies are never trusted.
- **404 instead of 403 for hidden resources.** A student asking for another class's assignment receives "not found", so the existence of the resource is not revealed.
- **Derived rather than stored states.** Only `DRAFT`/`PUBLISHED` are stored; `OPEN`, `CLOSED`, `PENDING` and `OVERDUE` are computed from the deadline and the clock, so they can never become stale.
- **Batch statistics.** Teacher statistics are computed with one grouped query over all visible assignment IDs (`summarizeByAssignmentIds`) instead of one query per assignment.

CHAPTER 6 – ACADEMICS & ASSIGNMENTS MODULE: DESIGN AND IMPLEMENTATION

6.1 Development in Two Phases

Phase 1 (PR #7, commit `22a5cf4`, 9 October). Because the backend did not yet exist, I first built the complete academics user interface — page, tabs, subject cards, filters, assignment list and details — against fictional data in `mockAcademicsData.js`. This allowed the layout and interactions to be reviewed early and fixed the data shapes that the backend would later have to return.

Phase 2 (PR #19, commit `a53614c`, 10 October). I implemented the backend, database tables and tests, and rewrote the frontend to use the live API. Mock data was reduced to a read-only, development-only demo that is never used for a real signed-in session: `loadAcademics()` loads it only when the user is in demo mode *and* `import.meta.env.DEV` is true, and every result carries `source: 'mock' | 'api'` so that the UI can label it with a "Demo data" badge.

6.2 Use Cases

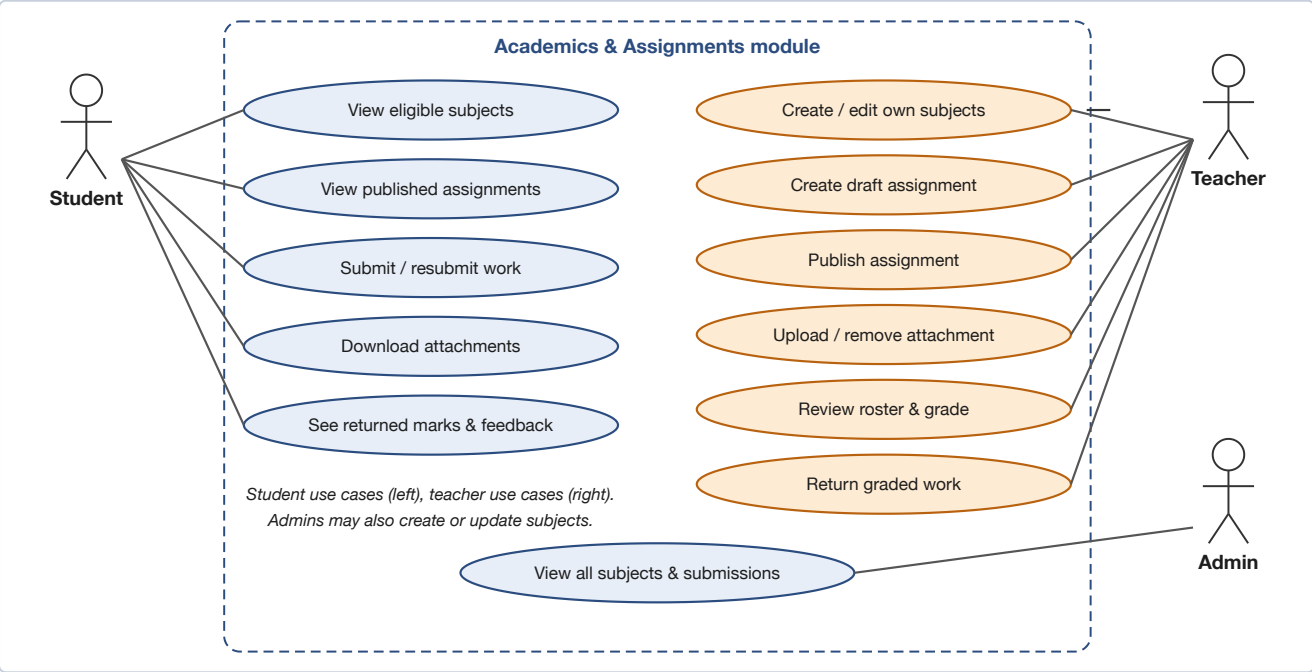


Figure 6.1 — Use-case diagram of the Academics & Assignments module.

6.3 Role-Based Access Rules

All fine-grained rules are concentrated in `AcademicAccessService`. A teacher "owns" a subject when `subject.teacher.id` equals their user ID. A student is "eligible" when the subject's department matches their profile and, where the subject specifies them, course, year and section also match (comparison is trimmed and case-insensitive). The same predicate is mirrored in the JPQL queries `SubjectRepository.findForStudent` and `StudentProfileRepository.findEligibleForAudience`, and a comment in the code requires them to be kept in sync.

Table 6.1 — Permission matrix enforced by the backend

Operation	Student	Teacher	Admin
List / view subjects	Eligible only	Own only	All
Create / update subject	No (403)	Own; self as teacher	Yes; chooses teacher
View assignments	Published, eligible	Own subjects (incl. drafts)	All (read-only)
Create, edit, publish, delete assignment	No	Own subjects only	No
Submit / resubmit	Own, before deadline	No	No

View roster of submissions	No (403)	Own subjects	All
Grade and return	No	Own subjects only	No
Download submission file	Own submission	Own subjects	All

6.4 Assignment and Submission Lifecycles

New assignments are saved as `DRAFT` unless the teacher selects "publish". Publishing sets `published_at`, requires a deadline in the future, and cannot be undone. Editing preserves publication state; changing the deadline to a past time, lowering maximum marks below the highest mark already awarded, or moving an assignment that already has submissions to another subject are all rejected. An assignment can be deleted only while it has no submissions, and its attachment file is then removed after the transaction commits.

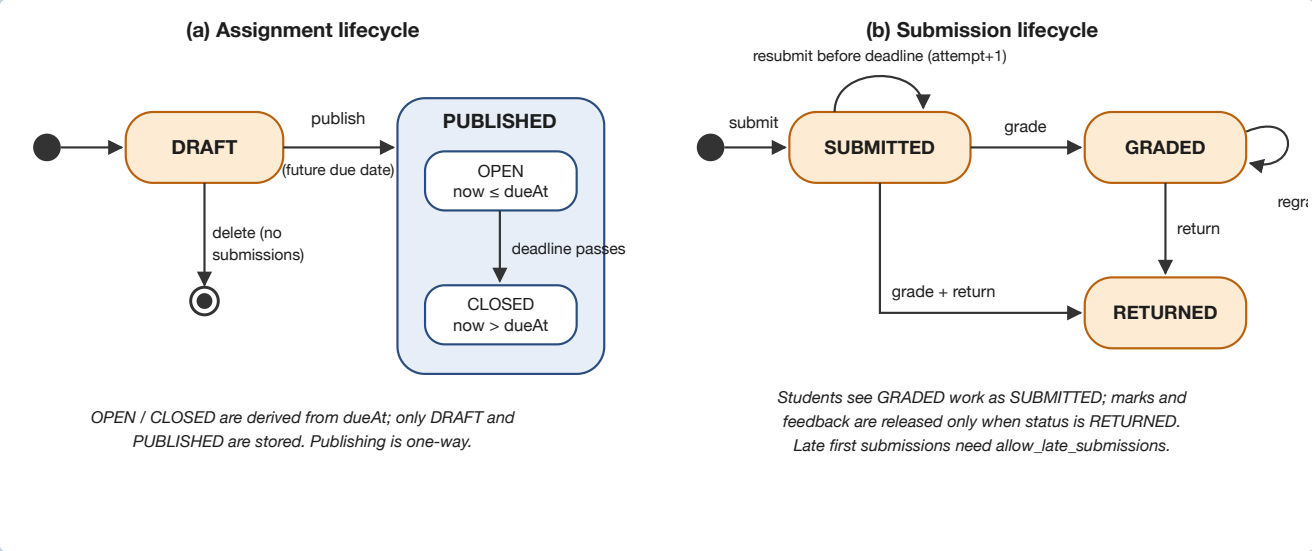


Figure 6.2 — State diagrams for (a) assignments and (b) submissions, as implemented in `AssignmentService`, `SubmissionService` and `AcademicMapper`.

For a student, each assignment is given a personal status: `PENDING` or `OVERDUE` before any submission (depending on the clock), `SUBMITTED` after submitting, and `GRADED` only once the teacher has returned the work. The mapper deliberately reports an internally `GRADED` submission as `SUBMITTED` to the student and withholds marks and feedback until release, which lets a teacher mark a whole class before returning results together.

6.5 Submission, Grading and Roster Workflow

Figure 6.3 traces a student submission end to end. Validation happens twice: the browser checks file type and size for fast feedback, and the server repeats every check. A first submission is rejected after the deadline unless `allow_late_submissions` is set, in which case it is accepted and flagged `late`. A submission must contain a written response (at most 20,000 characters) or a file. Two simultaneous submissions from the same student are caught by the unique key and reported as `409 Conflict` rather than a server error.

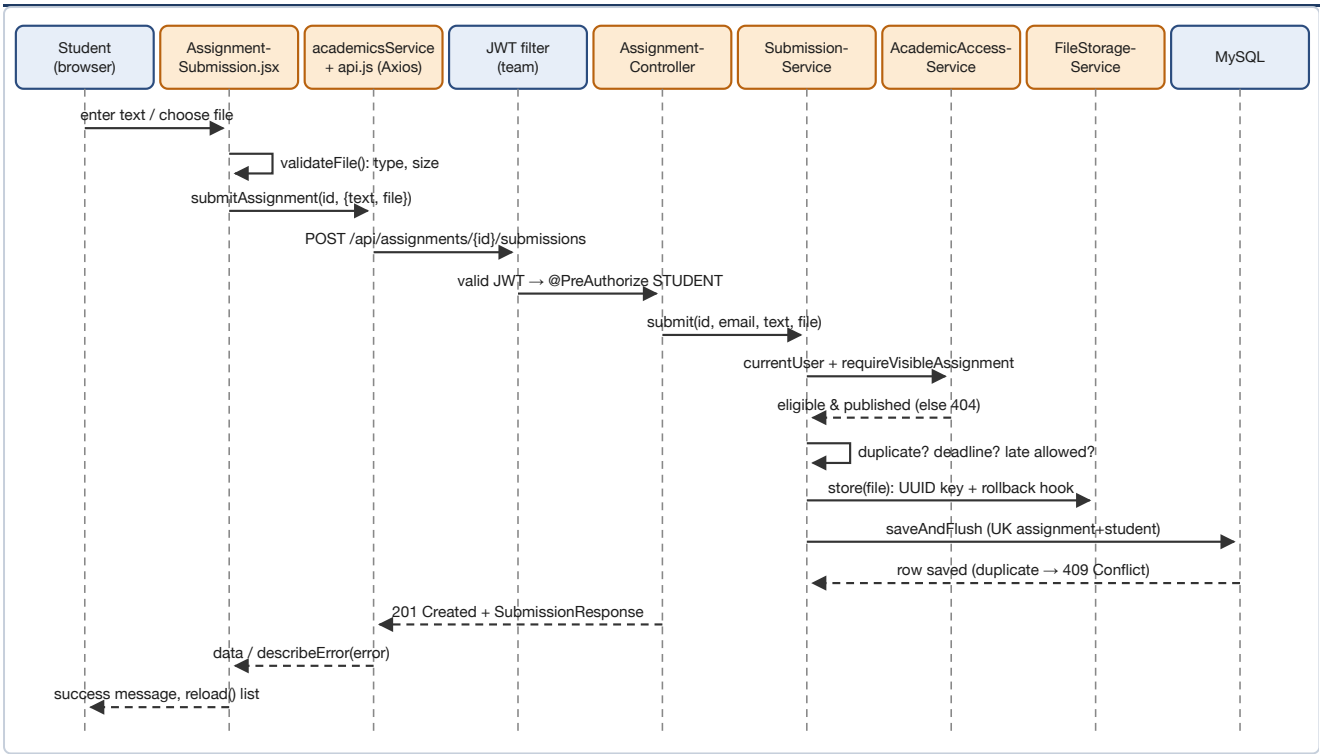


Figure 6.3 — Sequence diagram: a student submits an assignment with an attached file. Dashed arrows are responses.

Resubmission (`PUT /api/submissions/{id}`) replaces the text, optionally replaces or removes the file, increments `attempt_number` and resets the `late` flag. It is refused once the deadline has passed or the work has been graded. Grading (`POST /api/submissions/{id}/grade`) is restricted to the subject's teacher, checks that marks lie between zero and the maximum, records `graded_by` and `graded_at`, and — if `returnToStudent` is true — moves the submission to `RETURNED`, which cannot be reversed.

The grading roster (`GET /api/assignments/{id}/submissions`) lists every student whose profile matches the subject's audience, plus anyone who submitted but no longer matches. Students without a submission appear as `NOT_SUBMITTED`, so teachers see who is missing; the roster can be filtered by pending, submitted, graded or late.

6.6 Secure File Handling and Error Handling

`AcademicFileStorageService` keeps attachments and submission files in one configured directory (default `uploads/academics`, which I added to `.gitignore`). Files are saved as `<UUID>.<ext>`, so user-supplied names never influence the path; only allow-listed extensions are accepted and the content type is derived from the extension, not trusted from the client; size is limited to 10 MB by default; storage keys must match a strict UUID pattern and resolve inside the root directory, preventing path traversal; and a transaction-synchronisation hook deletes a newly written file if the transaction rolls back, while replaced files are deleted only after commit.

Rule violations raise typed exceptions that the global handler maps to consistent JSON errors: 400 for invalid input such as a past deadline, 403 for a wrong role, 404 for missing or hidden resources, 409 for conflicts such as a second submission or deleting an assignment with submissions, and 413 for oversized uploads. I added the 409 and 413 handlers to the handler created by the authentication module.

CHAPTER 7 – FRONTEND, DASHBOARD AND USER INTERFACE CONTRIBUTIONS

7.1 Shared Dashboard Shell (PR #2)

Commit `3dce324` introduced the shell that every signed-in page uses. `DashboardLayout` combines a fixed sidebar (`AppSidebar`) on large screens, a slide-in drawer (`MobileNavigation`) on small screens, a header with page title, theme toggle, notifications and user menus (`AppHeader`, `HeaderMenus`), a skip-to-content link, and a banner shown only in development demo mode. Sidebar links, page titles and dashboard shortcuts are defined once in

`config/navigation.js`, so teammates added their modules by editing one configuration file. `ThemeContext` stores the light/dark preference in `localStorage` and falls back to the operating-system setting, and a small inline script in `index.html` applies the theme before first paint to avoid a flash.

7.2 Routing and Guards

I wrote the original `ProtectedRoute` and `RequireRole` components; Aman Kumar Chauhan later extended `ProtectedRoute` with the redirect to `/login` and added `PublicOnlyRoute`. `RequireRole` hides pages from the wrong roles, but — as its code comment states — it is a user-experience convenience, not a security boundary; the backend checks every request. In PR #18 I converted all signed-in pages to `React.lazy` imports with a `Suspense` fallback in the layout, so that, for example, the academics page is downloaded only when it is opened.

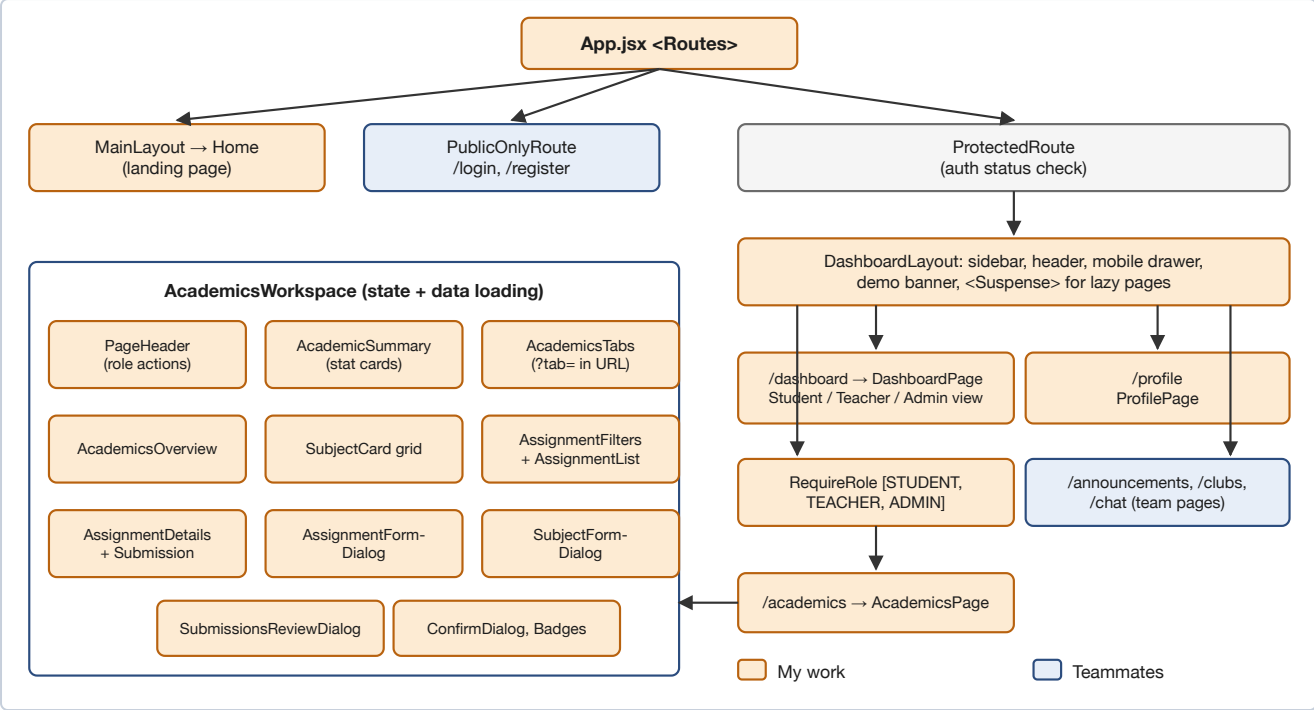


Figure 7.1 — Route, layout and component structure of the signed-in frontend, from `App.jsx` and `AcademicsPage.jsx`.

7.3 Role-Specific Dashboards

`DashboardPage` selects `StudentDashboard`, `TeacherDashboard` or `AdminDashboard` from the signed-in user's role — never from the URL — and remounts it when the role changes. Each dashboard is assembled from reusable pieces: `WelcomeSection`, `StatCard` / `StatGrid`, a generic `DataSection` that handles loading skeletons, empty states and a data-source badge, and list components such as `DeadlineList`, `PendingReviewList`, `AnnouncementList` and `UpcomingEvents`. A shared `useAsyncData` hook loads data, exposes `loading/success/error` and a `reload` function, and ignores results that arrive after the component has unmounted. In development, `?demoState=loading|empty|error` previews each state.

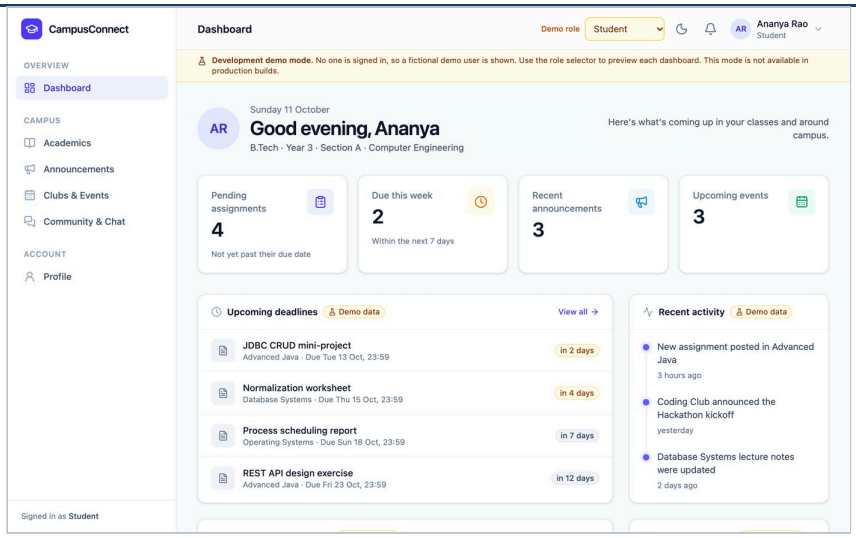


Figure 7.2 — Student dashboard rendered by the development build in demo mode on 11 October 2026. The banner and "Demo data" badges show that the data is fictional.

7.4 Academics User Interface

`AcademicsPage` keeps all page state in an inner `AcademicsWorkspace` component that is keyed by role, so switching roles resets it cleanly. The active tab (overview, subjects, assignments) is stored in the URL query string, which makes tabs bookmarkable and preserves them on refresh. Header actions depend on role: students get "Browse subjects" and "View pending work" (which pre-filters the list), teachers get "New subject" and "New assignment" (disabled until they own a subject), and admins get "New subject". Dialogs are driven by a single `dialog` state object, so only one is open at a time and closing a nested dialog returns to the assignment details.

Figure 7.1 lists the components. `AcademicsTabs` implements the ARIA tab pattern with Arrow/Home/End navigation; `AssignmentList` marks work due within 48 hours and shows submission progress; `AssignmentDetails` and `AssignmentSubmission` provide publishing and deletion for teachers and submission, revision and returned grades for students; and `SubmissionsReviewDialog` offers roster filters, marks and feedback entry with "save" or "save and return".

7.5 Validation, Loading and Error States

Forms validate before sending: a title is required (maximum 200 characters), maximum marks must be a whole number from 1 to 1000, and a changed deadline must be in the future. Files are checked against the same extension list and 10 MB limit as the server. Inputs set `aria-invalid` and link error text with `aria-describedby`. While loading, skeleton lists are shown; failures show an error card with a retry button; empty lists show role-specific guidance. Converting `datetime-local` input values to ISO instants (and back with `toLocalInputValue`) keeps deadlines correct in the user's time zone.

I also built the public landing page (`9646c18`) from twelve components in `components/landing/`, including the hero, problem statement, features, academics and community showcases, "how it works" steps, a dashboard preview and footer.

CHAPTER 8 – API INTEGRATION AND BACKEND COORDINATION

8.1 Shared HTTP Client

I created `services/api.js` in the initial scaffold as the single Axios instance whose base URL comes from `VITE_API_BASE_URL`. Teammates later added the request interceptor that attaches the bearer token and the 401 handling. Because every academics call goes through this instance, my module inherited authentication without any module-specific token code.

8.2 Academics Endpoints

Table 8.1 — REST endpoints of the Academics module (22 in total) and their frontend callers

Method and path	Role check	Frontend function
`GET /api/subjects`, `GET /api/assignments`	Signed-in	`loadAcademics()`
`GET /api/subjects/{id}`, `…/{id}/assignments`, `GET /api/assignments/{id}`	Service check	— (available to other clients)
`GET /api/subjects/teachers`	ADMIN	`fetchTeacherOptions()`
`POST /api/subjects`, `PUT /api/subjects/{id}`	TEACHER, ADMIN	`createSubject()`, `updateSubject()`
`POST`, `PUT`, `DELETE /api/assignments[/{id}]`, `POST …/{id}/publish`	TEACHER	`create/update/delete/publishAssignment()`
`POST`, `DELETE`, `GET /api/assignments/{id}/attachment`	TEACHER (GET: service check)	`upload/remove/downloadAssignmentAttachment()`
`POST /api/assignments/{id}/submissions` (multipart)	STUDENT	`submitAssignment()`
`GET /api/assignments/{id}/submissions?status`	TEACHER, ADMIN	`fetchAssignmentSubmissions()`
`GET /api/submissions/mine`, `…/{id}`, `…/{id}/file`	STUDENT / service check	`downloadSubmissionFile()`
`PUT /api/submissions/{id}` (multipart), `POST …/{id}/grade`	STUDENT / TEACHER	`resubmitAssignment()`, `gradeSubmission()`

The frontend loads subjects and assignments in parallel with `Promise.all` and performs filtering and sorting locally, because one class's assignment list is small; the server-side query parameters remain available for other clients. After each mutation the page calls `reload()` so that derived statistics stay consistent with the server.

8.3 Multipart Uploads and Authenticated Downloads

Submissions are sent as `multipart/form-data` with optional `textResponse`, `file` and `removeFile` parts. File downloads require the bearer token, so a plain link cannot be used; instead the file is fetched as a `Blob`, turned into a temporary object URL, downloaded through a hidden anchor and released after one second. On the server, `FileDownloadResponses` sets safe download headers. Because error responses to blob requests also arrive as blobs, `describeError()` parses them as JSON to show the server's message, prefers the first field-level validation message, and replaces Spring's generic "Access is denied" with a friendlier sentence.

8.4 Coordination with Teammates' Modules

- **Authentication:** my controllers rely on the team's JWT filter to populate `Authentication`; I use only `authentication.getName()` (the e-mail) and load the user in the service.
- **Profiles:** eligibility depends on `department`, `course`, `year` and `section` in Aman's `StudentProfile`; I added read-only audience queries to its repository rather than changing the entity.
- **Dashboard contracts:** `docs/dashboard.md` documents the data shapes each dashboard widget consumes, so module owners could replace mock sections with API calls. The announcements widget was switched to live data in this way by a teammate; the academics widgets have not yet been switched.
- **Registration:** in PR #21 I added a required `college` field to registration (DTO, entity, column, service check and a select on the register page) so that accounts can only be created for Arya College of Engineering & IT,

with matching tests in `AuthServiceTest` and `AuthControllerTest`.

- **Deployment:** the multi-stage `Dockerfile` builds the SPA with same-origin paths (`/api`, `/ws/sockjs`) and packages it inside the Spring Boot JAR. `SpaWebConfig` serves static files and falls back to `index.html` for client routes, while `SecurityConfig` now requires authentication for `/api/**` and permits the static site.

CHAPTER 9 – GIT WORKFLOW, BRANCHING, MERGING AND TEAM COLLABORATION

9.1 Repository Set-up and Conventions

I created the repository on 8 October 2026 (`3512458`) and, in the "base created" commit (`e451da8`), established the shared foundation: the Maven project with package skeleton, CORS and a temporary permit-all security configuration, the Vite/React/Tailwind frontend with the shared Axios client, `.env.example` files, and a README describing set-up and collaboration rules — never commit to `main` directly, sync before work, use feature branches, keep commits focused, require review, never commit secrets, and coordinate changes to shared files.

9.2 Branches and Pull Requests

Git records 29 commits under my identity: 12 feature or fix commits, 11 pull-request merges that I performed as repository owner (#1, #2, #3, #7, #8, #9, #11, #18, #19, #20, #21), 4 merges that synchronised feature branches with `main`, and 2 commits created by `git stash`. Excluding `package-lock.json`, my non-merge commits add about 12,600 lines, a figure that includes tests and generated scaffolding. All of my feature branches have been merged into `main`.

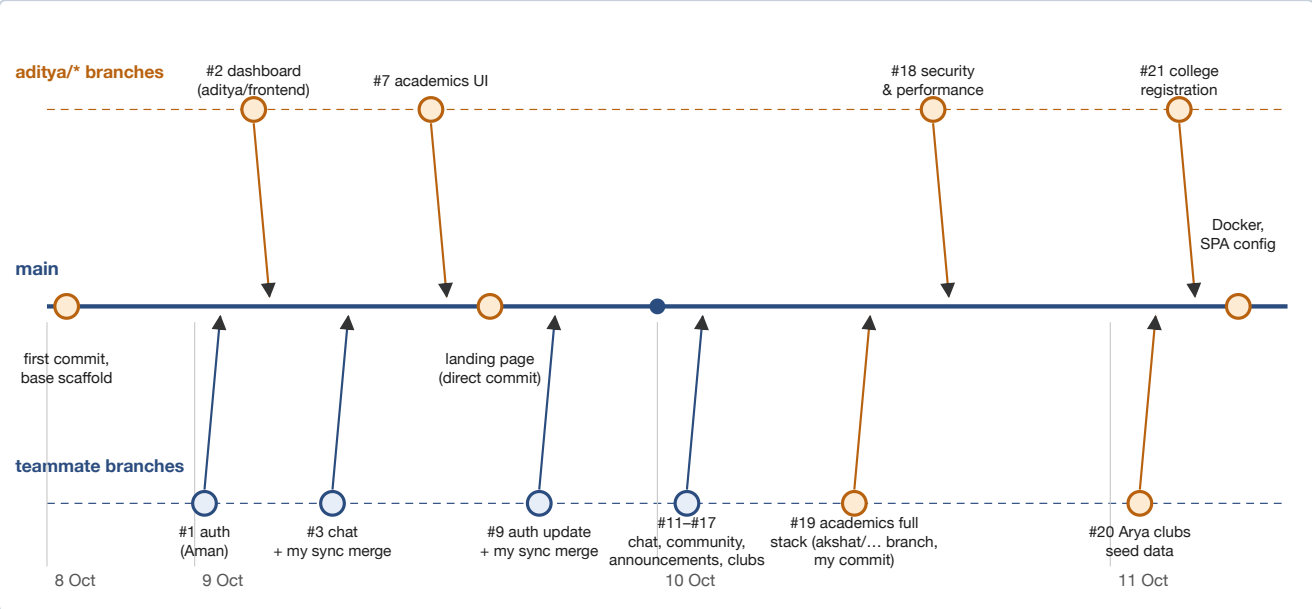


Figure 9.1 — Branch and pull-request timeline reconstructed from `git log --all --graph`. Orange markers are my commits or merges.

9.3 Conflict Resolution and Synchronisation

Table 9.1 — Integration merges I performed (combined diffs show manually resolved hunks)

Commit	Merge	Files reconciled and purpose
`f05725f`	`main` → `ayushman-feature`	`App.jsx`, `AuthContext.jsx`, `package.json`: combined the chat routes and dependencies with the new dashboard shell before PR #3. A stash ("preserve chat dependencies and Vite config") protected uncommitted work during this step.
`9d5d2a7`	`origin/main` → `aman-auth-profile`	`App.jsx`, `AuthContext.jsx` and landing components: joined the login/register flow and token persistence with the dashboard, academics and chat routes before PR #9.

`a17f37f`	`main` → `akshat/assignment-academics`	`application.properties` : kept both the clubs settings and the new academics upload settings before PR #19.
`dab9102`	`origin/main` → `aditya/fix-security-perf`	Re-based the security fix on the merged academics code before PR #18.

9.4 Security and Hygiene Fixes (PR #18)

While integrating, I found that defaults in `application.properties` contained a real-looking JWT signing secret and a default administrator e-mail and password, and that the provisioning runner reset the administrator's password on every start. Commit `ec9b9e2` made `JWT_SECRET` mandatory, disabled administrator provisioning unless explicitly configured, stopped overwriting existing passwords, removed a stale `backend/bin` copy of the project, replaced a paged query that ran an unnecessary count query for chat previews, and added lazy loading of pages. Commit `2572458` later ordered the provisioning runner before the clubs seeder.

9.5 Reflection on the Workflow

The history also shows where I did not follow our own rules: the landing page (`9646c18`) and the Docker commits were made directly on `main` , and branch names did not follow the README's `feature/<module>` pattern. All 21 pull requests were merged within four days (8–11 October), which left little time for the teammate review that the README requires. These are lessons recorded in Chapter 12.

CHAPTER 10 – TESTING, DEBUGGING AND VERIFICATION

10.1 Testing Strategy

The academics module is covered by fast, isolated tests written with JUnit 5 and Mockito. Service tests mock the repositories and inject a fixed `Clock` , so deadline behaviour ("one minute before", "one minute after") is deterministic. Controller tests use `MockMvcBuilders.standaloneSetup` to check HTTP status codes, request validation, multipart handling and download headers without starting the application. A shared `AcademicTestData` builder creates consistent users, profiles, subjects and assignments.

10.2 Verification Run (11 October 2026)

The complete backend suite was executed with `mvn test` using Temurin JDK 21.0.12, on a copy of the `backend/` sources at commit `2572458` . Results were read from the Surefire reports.

Table 10.1 — Academics test classes and results

Test class	Tests	Focus	Result
`AssignmentServiceTest`	23	Drafts, publishing, ownership, deadlines, deletion, statistics, attachments	23 passed
`SubmissionServiceTest`	23	Deadlines, late work, duplicates, resubmission, grading, hidden marks	23 passed
`AssignmentControllerTest`	15	Status codes, validation, multipart, download headers	15 passed
`SubjectServiceTest`	14	Audience eligibility, admin teacher assignment, uniqueness	14 passed
`SubjectControllerTest`	9	Endpoints, validation, role restrictions	9 passed
`SubmissionControllerTest`	8	Own submissions, resubmission, grading validation, 403/404 mapping	8 passed
`AcademicAccessServiceTest`	6	Ownership and eligibility predicates	6 passed
`AcademicFileStorageServiceTest`	6	Generated names, allow-list, size, tampered keys	6 passed
Academics total	**104**	8 classes written in commit `a53614c`	0 failures
Whole backend suite	227	26 classes including teammates' modules	0 failures

The registration change in PR #21 extended `AuthServiceTest` (12 tests) and `AuthControllerTest` (5 tests), both of which passed in the same run. The frontend production build (`vite build`, Vite 8.3.4) also completed without errors, transforming 2,218 modules and emitting the academics page as a separate 57.28 kB chunk (15.29 kB gzip), which confirms the lazy-loading change.

Representative test methods include `submit_afterDeadline_isRejectedUnlessLateAllowed`, `submit_concurrentDuplicate_isReportedAsConflict`, `getSubmission_enforcesOwnershipAndHidesUnreturnedGrades`, `updateAssignment_maxMarksBelowAwardedMarks_isRejected`, `getAssignment_studentCannotSeeDraft` and `load_rejectsTamperedStorageKeys`.

10.3 Debugging Notes

Two problems found during integration are visible in the history. First, error bodies for file downloads were lost because Axios returns them as blobs; this led to the blob-aware `describeError()`. Second, a stale duplicate of the backend under `backend/bin` (an IDE build output) had been committed and could be confused with the real sources; it was removed in PR #18.

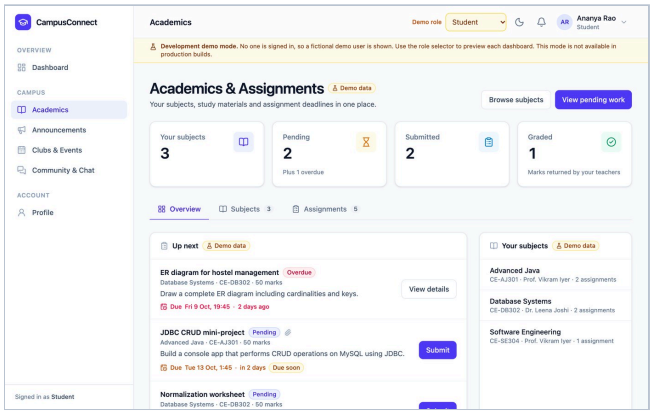
10.4 Pending Verification

- **Pending** End-to-end tests against a running MySQL database and backend (no repository tests use a real database).
- **Pending** Automated frontend tests — the frontend has no test runner configured.
- **Pending** Manual browser testing of the live API flow; screenshots in this report come from demo mode only.
- **Pending** Verification of the Docker image and of any hosted deployment.

CHAPTER 11 – IMPLEMENTATION RESULTS, CURRENT STATUS AND LIMITATIONS

11.1 Results

Figures 11.1 and 11.2 show the academics page for a student and for a teacher. They were rendered by the project's development server in demo mode, which uses fictional people and subjects; they demonstrate the implemented user interface, not real usage data.



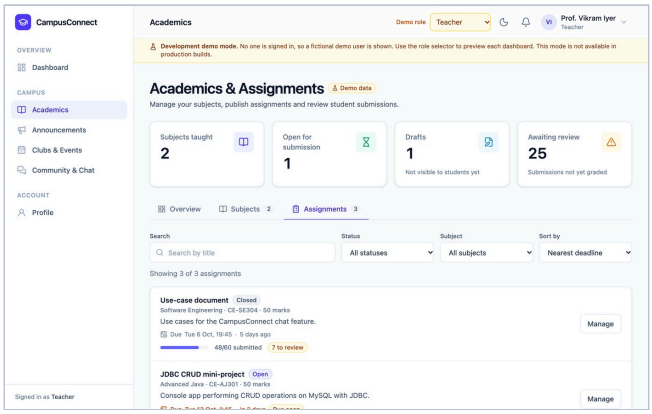


Figure 11.1 — Academics page, student view (left): summary cards, tabs, upcoming work with Overdue/Pending/Due-soon badges and the student's subjects.

Figure 11.2 — Academics page, teacher assignments tab (right): search, status, subject and sort filters, Open/Closed status and submission progress.

11.2 Status Summary

Table 11.1 — Status of functionality within my scope

Functionality	Status
Academics backend: subjects, assignments, submissions, grading, files, access rules	**Implemented** — unit and controller tests pass

Academics frontend wired to live API	**Implemented** — builds; live end-to-end run **pending**
Dashboard shell, navigation, theme, guards	**Implemented**
Student/teacher dashboard assignment and review widgets	**Partial** — still use labelled mock data
Admin dashboard user statistics, activity feeds, notifications bell	**Partial** — mock data / not connected
College-restricted registration	**Implemented** — tests pass
Docker single-origin packaging	**Implemented, unverified**
Notifications, plagiarism checks, calendar view, grade export	**Planned**

11.3 Limitations

- The dashboard's assignment deadlines, pending reviews and related counts are not yet read from the academics API, so they can disagree with the academics page.
- Student–subject membership is inferred from profile fields; there is no explicit enrolment, elective or section-change history.
- Files are stored on the server's local disk, which is lost if a container is replaced unless a volume is mounted; there is no virus scanning.
- One submission per student per assignment; no group work, rubrics or multiple graders.
- The schema is maintained both in `schema.sql` and by Hibernate's `ddl-auto=update`; a migration tool would be safer.
- Subject codes are globally unique, which is not suitable for multi-college operation without an institution scope.

CHAPTER 12 – CHALLENGES AND LEARNING OUTCOMES

12.1 Challenges and Solutions

Table 12.1 — Challenges encountered and how they were addressed

Challenge	Solution and evidence
Building the UI before the backend existed	Phase 1 used labelled mock data with the same shapes later returned by the API; phase 2 confined mock data to development demo mode (`22a5cf4` → `a53614c`).
Keeping access rules consistent across many endpoints	One `AcademicAccessService` used by all services, mirrored by JPQL audience queries and covered by its own tests.
Time-dependent rules (deadlines, overdue, late)	Derived statuses computed from an injected `Clock`; tests use a fixed clock.
Hiding marks until release	Separate student and staff mappings in `AcademicMapper`; a test asserts unreturned grades are withheld.
Safe file uploads	UUID names, extension allow-list, size limits, path checks and transaction-aware clean-up.
Concurrent duplicate submissions	Database unique key plus translation of `DataIntegrityViolationException` into 409.
Merging five parallel workstreams into shared files	Synchronisation merges with manual conflict resolution in `App.jsx`, `AuthContext.jsx` and properties files; `git stash` to protect local work.
Secrets in default configuration	Mandatory `JWT_SECRET`, opt-in admin provisioning, no password overwrite (`ec9b9e2`).

12.2 Technical Learning Outcomes

- Designing a relational schema with constraints that protect business rules, and mapping it with JPA entities and fetch-aware repository queries.
- Applying layered Spring Boot design, method-level security, transactional services and consistent exception-to-status mapping.

- Practical secure-coding habits: object-level authorisation, not revealing hidden resources, and defensive file handling.
- Building a component-based React interface with shared hooks, URL-driven state, accessible tabs and dialogs, and responsive layouts with Tailwind CSS.
- Writing deterministic unit tests with Mockito and MockMvc, and reading Surefire reports.
- Packaging a full-stack application with a multi-stage Docker build.

12.3 Professional Learning Outcomes

- **Integration is a job of its own.** As repository owner I learned that merging is easier when interfaces (navigation config, dashboard data contracts) are agreed early and documented.
- **Follow the rules you write.** Committing directly to `main` and merging pull requests quickly saved time but reduced the opportunity for review; in future I would protect `main` and require an approving review before merging.
- **Honest status reporting.** Labelling mock data visibly in the UI and reporting partial features as partial made reviews more productive and this report more accurate.
- **Clear attribution.** Working on a branch named for another teammate showed me why commit authorship, branch naming and pull-request descriptions must make ownership clear.

CHAPTER 13 – FUTURE SCOPE AND CONCLUSION

13.1 Future Scope

1. **Live dashboard data:** replace the mock assignment, deadline and pending-review sections in `dashboardService.js` with calls to `/api/assignments` and the roster endpoint, as documented in `docs/dashboard.md`.
2. **End-to-end and frontend tests:** add integration tests against a test MySQL database and component/end-to-end tests for submission and grading flows, and run them in continuous integration.
3. **Explicit enrolment:** introduce an enrolment table to support electives and section changes, keeping profile matching as a default.
4. **Notifications:** notify students when an assignment is published or returned, reusing the header notification menu.
5. **Durable file storage:** move files to object storage or a mounted volume, and add malware scanning.
6. **Richer assessment:** rubrics, group submissions, grade export (CSV) and an academic calendar view.
7. **Multi-college readiness:** scope subjects and codes by institution, in line with the team report's roadmap.
8. **Schema migrations:** adopt a migration tool and disable automatic schema updates in production.

13.2 Conclusion

My contribution to CampusConnect turned the team report's planned "assignments and academic section" into a working, tested module. The Academics & Assignments module provides a complete workflow — subjects, draft and published assignments, attachments, submissions, resubmission, grading and the controlled release of marks — backed by three constrained MySQL tables and 22 role-checked REST endpoints. Its central design choices are that identity always comes from the authenticated user, that one service defines who may see and do what, and that time-dependent states are derived rather than stored. One hundred and four unit and controller tests for the module, and the full suite of 227 backend tests, passed on 11 October 2026.

Beyond the module, I built the dashboard shell and landing page that frame every other module, created and maintained the shared repository, merged and reconciled my teammates' work, removed insecure default configuration, and prepared single-origin Docker packaging. The work is not finished: the dashboard still shows demonstration data for academic widgets, and end-to-end, frontend and deployment testing remain pending. These limitations are stated openly so that the next iteration can address them. The project has given me practical experience of full-stack development, secure design and, above all, of building software together with a team.

REFERENCES

[1] Meta Open Source, “React Documentation,” https://react.dev/ (accessed Oct. 2026).

[2] Remix Software, “React Router Documentation,” https://reactrouter.com/ (accessed Oct. 2026).

[3] Vite Team, “Vite Guide — Building for Production,” https://vite.dev/guide/build (accessed Oct. 2026).

[4] Tailwind Labs, “Tailwind CSS Documentation — Dark Mode and Responsive Design,” https://tailwindcss.com/docs (accessed Oct. 2026).

[5] Axios Project, “Axios Documentation — Request Config and Interceptors,” https://axios-http.com/docs/intro (accessed Oct. 2026).

[6] VMware Tanzu / Broadcom, “Spring Boot Reference Documentation,” https://docs.spring.io/spring-boot/ (accessed Oct. 2026).

[7] Spring Security Team, “Method Security (@PreAuthorize),” Spring Security Reference, https://docs.spring.io/spring-security/reference/servlet/authorization/method-security.html.

[8] Spring Team, “Spring Data JPA Reference Documentation,” https://docs.spring.io/spring-data/jpa/reference/.

[9] Spring Team, “Transaction Management and TransactionSynchronization,” Spring Framework Reference, https://docs.spring.io/spring-framework/reference/data-access/transaction.html.

[10] Oracle, “MySQL 8.0 Reference Manual — CHECK Constraints and FOREIGN KEY Constraints,” https://dev.mysql.com/doc/refman/8.0/en/.

[11] JUnit Team, “JUnit 5 User Guide,” https://junit.org/junit5/docs/current/user-guide/.

[12] Mockito Project, “Mockito Documentation,” https://site.mockito.org/.

[13] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, May 2015.

[14] OWASP Foundation, “File Upload Cheat Sheet,” OWASP Cheat Sheet Series, https://cheatsheetseries.owasp.org/.

[15] OWASP Foundation, “Authorization Cheat Sheet” and “Insecure Direct Object Reference Prevention Cheat Sheet,” OWASP Cheat Sheet Series.

[16] W3C, “ARIA Authoring Practices Guide — Tabs Pattern and Dialog (Modal) Pattern,” https://www.w3.org/WAI/ARIA/apg/.

[17] S. Chacon and B. Straub, *Pro Git*, 2nd ed., Apress, 2014; Git documentation for `git merge` and `git stash`, https://git-scm.com/doc.

[18] Docker Inc., “Multi-stage Builds,” Docker Documentation, https://docs.docker.com/build/building/multi-stage/.

[19] Aditya Gour, Anjali Kumari, Aayushman Pathak, Akshat Varma and Aman Kumar Chauhan, “CampusConnect: Student Academic & Communication Portal,” Project Report, Dept. of CSE, Arya College of Engineering & I.T., Jaipur, Session 2026–27.

[20] CampusConnect source repository, https://github.com/Adit841/CampusConnect, branch `main` at commit `2572458`, including `docs/architecture.md` and `docs/dashboard.md`.

APPENDIX A – COMMIT EVIDENCE AND VERIFICATION LOG

A.1 Branches and Commits Inspected

My feature and fix commits are listed with their pull requests in Table 2.1 and the merges in Table 9.1. The branches inspected were `aditya/frontend`, `aditya/academics-assignments`, `akshat/assignment-academics`, `aditya/fix-security-perf`, `aditya/registration-college`, `aditya/add-real-arya-clubs` (local) / `dheeraj/add-real-arya-clubs` (remote), together with the teammates' `aman-auth-profile`, `ayushman-feature` and `anjali-announcements` branches.

A.2 Verification Log

Repository state	`main` at `2572458`; all `aditya/*` branches contained in `main`.
Backend tests	`mvn -B -q test`, JDK 21.0.12 — 227 run, 0 failures, 0 errors, 0 skipped.
Frontend build	`vite build` (Vite 8.3.4) — success, 2,218 modules transformed.
Screenshots	Development server, demo mode (fictional data), 1280 × 800 and 390 × 844 viewports, 11 October 2026.
Not executed	Live MySQL end-to-end flow, Docker image build, hosted deployment.

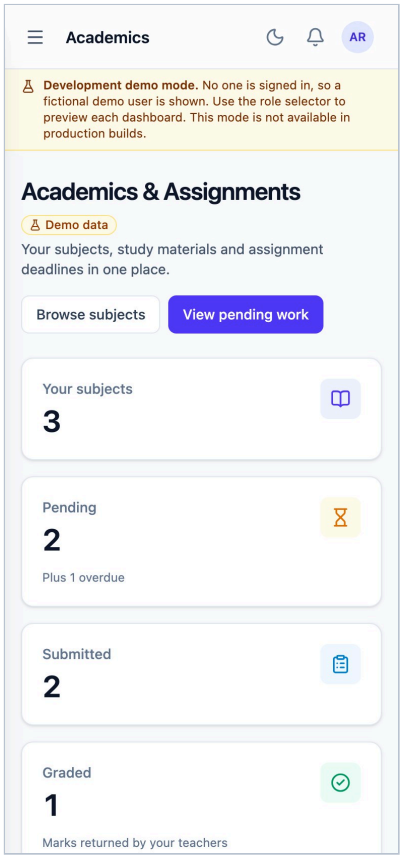


Figure A.1 — Academics page at a 390 px mobile viewport (development demo mode): collapsed navigation, stacked actions and summary cards.